TÉCNICO LISBOA

Distributed and Parallel Computing

Serial Report

Pedro Rio
MSc Data Science and Engineering
Instituto Superior Técnico
97241

Sebastião Maia Serqueira
MSc Data Science and Engineering
Instituto Superior Técnico
97108

Eduardo Machado
MSc Data Science and Engineering
Instituto Superior Técnico
97133

1 Program Overview

The problem is a matrix-factorisation recommender: a sparse user–item matrix A is approximated by the product of two dense factors, $A \approx L \cdot R$, where L is *users* $\times$ *features* and R is *features* $\times$ *items*. The factors are fitted by gradient descent over the observed (non-zero) entries, and the final dense product $B = L \cdot R$ yields one recommendation per user. The serial implementation is divided into four parts.

1.1 ReadInput

A single pass over the instance file parses the header (iterations, convergence coefficient, features, and the *users*, *items*, *non-zeros* triple) and then the non-zero entries. Matrix A is allocated as one flat row-major buffer and the non-zero coordinates are kept in parallel index arrays for the gradient loop.

1.2 InitialLR

The provided routine seeds a pseudo-random generator and fills L and R. It is inherently sequential: the values depend on the order of the random draws, so the fill order is part of the specification.

1.3 UpdateLR

For each non-zero (i, j) a predicted score is the inner product of row i of L and column j of R; the residual δ is the difference between the observed value and the prediction. Each factor is then nudged along the gradient of the squared residual, scaled by the convergence coefficient. The step uses a *snapshot* of L and R taken at the start of the iteration, and the whole routine is repeated for the requested number of iterations.

1.4 FilterFinalMatrix

The dense product $B = L \cdot R$ is formed, the already-known non-zero positions are masked out, and for each row the column of maximum value is selected and streamed to standard output as that user's recommendation.

2 Implementation Highlights

Although the program is single-threaded, several decisions shape both its absolute speed and how cleanly it later parallelises.

2.1 Cache-aware $B = L \cdot R$

The dense product dominates the final stage. With row-major storage, the natural i, j, k loop nest reads

$R[k][j]$ down a column — stride *items* — on every step of the innermost loop, thrashing the cache. We instead use the i, k, j order: the scalar $L[i][k]$ is loop-invariant in j and is hoisted, and the inner loop sweeps $B[i][j]$ and $R[k][j]$ *together with unit stride*.

```
for (i = 0; i < users; i++) {
  for (j = 0; j < items; j++)
    B[i*items + j] = 0;
  for (k = 0; k < features; k++) {
    double lik = L[i*features + k];
    for (j = 0; j < items; j++)
      B[i*items + j] += lik * R[k*items + j];
  }
}
```

Both operands of the hot loop are now contiguous, which keeps the data in cache and lets the compiler vectorise the accumulation (SIMD). The arithmetic is identical to the textbook order; only the access pattern changes.

2.2 Jacobi snapshot in UpdateLR

Each iteration first copies L and R into `StoreL`/`StoreR`; every update then *reads the snapshot and writes the live factors*. This makes the iteration a Jacobi step (all updates see the same starting point) rather than a Gauss–Seidel one, so the result does not depend on the order in which non-zeros are visited. Index arithmetic into the flat buffers is widened to `size_t` to avoid 32-bit overflow on large instances.

2.3 Robust input handling

`readInput` validates as it parses: the file must open, the header must be complete, the dimensions must be positive, every non-zero index must fall inside $[0, \mathit{users}) \times [0, \mathit{items})$, and the number of non-zero lines must match the declared count. Any violation fails fast through a single error path instead of silently indexing out of bounds. Valid instances are unaffected; malformed ones stop with a diagnostic rather than corrupting memory.

2.4 Flat row-major layout

Every matrix is a single contiguous array addressed as `M[row*width + col]`. This minimises allocations and pointer chasing, gives the predictable stride patterns the cache optimisation relies on, and — as Section 3 notes — makes contiguous sub-blocks trivial to ship between processes later.

3 Implications for Parallel and Distributed Computing

The serial version is not merely a first draft to be discarded; the choices above are what make the parallel versions correct, fast, and tractable.

3.1 A correctness oracle

Because `initialLR` is deterministic, the serial program produces a fixed, reproducible set of recommendations. That output *defines* correctness: the OpenMP and MPI versions are validated by matching it bit-for-bit across every thread count, process count, and grid shape. Without a deterministic serial reference there is no cheap oracle, and floating-point reordering in the parallel code could not be distinguished from genuine bugs.

3.2 Dependencies dictate the strategy

The snapshot exposes the parallelism precisely. Within an iteration every residual is computed from read-only snapshot data, so the non-zeros are mutually independent — yet two non-zeros sharing a user touch the same row of L, and two sharing an item touch the same column of R. Those write conflicts are exactly the hazard that forces a per-thread reduction in OpenMP and an `Allreduce` of the gradient in MPI. Reading the serial loop tells you both *where* the parallelism is and *which* writes must be coordinated.

3.3 Locality is the shared lever

The kernel is memory-bound: it streams L and R far more than it computes. The i, k, j optimisation therefore does double duty — it speeds the serial run, and because the parallel versions are bandwidth-bound, the serial access pattern sets the *ceiling* they can reach. It also explains why added threads stop helping once a socket's memory bandwidth saturates, rather than at the core count. The practical rule: make the serial loop cache-friendly first, then parallelise.

3.4 An honest speedup baseline

Speedup is reported against the serial time, so an un-optimised baseline inflates the numbers. Tightening the serial kernel first means the parallel speedups measure real work, not a strawman — and it is why a one-thread OpenMP run does not sit above 1.0: it pays fork/join and reduction overhead against a tight serial for no parallelism in return.

Measured on an Apple Silicon laptop (macOS 15.6, 10 cores, 8 of them performance cores) by `compare/scripts/benchmark.py`, three repetitions with the minimum kept, every run verified against the instance's `.out` file. Every figure here traces to a row in `compare/data/comparison.{serial,omp}.csv` and is reproducible from this repository; the cluster figures in the OpenMP report were measured on different hardware and are not comparable with these.

Table 1: The serial baseline against a one-thread OpenMP run — **this laptop** (time, s).

fileName	serial	OpenMP, 1 thread	ratio
inst1000-1000-100-2-30.in	4.48	4.77	0.94
inst500-500-20-2-100.in	8.51	10.39	0.82
inst200-10000-50-100-300.in	6.68	7.57	0.88

The ratio runs 0.82–0.94: a one-thread parallel run is 6–22 % *slower* than the serial program it is built from. Quoting that column as 1.0 would silently credit the parallel version with work it never did.

3.5 The irreducible serial fraction

`initialLR` cannot be parallelised: the random draws must occur in order, or the factors — and thus the output — change. By Amdahl's law this sequential fraction bounds the achievable speedup; identifying it in the serial program fixes the parallel ceiling before a single thread is spawned.

3.6 Layout enables decomposition

Because the factors are contiguous and row-major, a block of user-rows or item-columns is a contiguous span of memory. That is what lets the MPI versions distribute L and R with simple `Scatterv`/`Bcast` calls and lets the 2D-grid variant own clean sub-blocks per process. A serial data-layout decision quietly determines how efficient the distributed partitioning can be.

3.7 Robust I/O becomes collective safety

Fail-fast validation matters more once there are many processes: if one rank reads a malformed file and aborts while the others wait in a collective, the job deadlocks. The serial habit of validating on read and stopping cleanly generalises directly to the MPI pattern of validating on the root, broadcasting the status, and aborting together — defensive serial code is what makes collective error handling safe.